

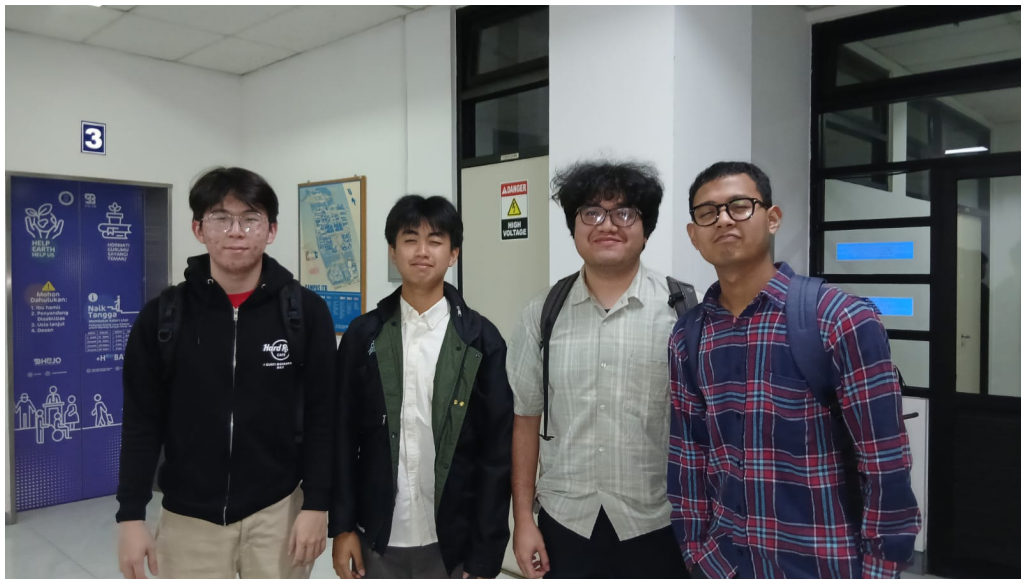
Tugas Besar

Milestone 2

IF2224 Teori Bahasa Formal dan Otomata

Lexical Analysis

Semester 2 Tahun 2025/2026



Disusun oleh:

Kai

Jonathan Kris Wicaksono	(13524023)
Vincent Rionarlie	(13524031)
Muhammad Aufar Rizqi Kusuma	(13524061)
Bryan Pratama Putra Hendra	(13524067)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



Daftar Isi

1	Pendahuluan	2
2	Dasar Teori	3
2.1	Teori Singkat	3
2.1.1	Parser dan Syntax Analysis	3
2.1.2	Parse Tree	4
2.1.3	Recursive Descent Parser	6
3	Perancangan dan Implementasi	8
4	Pengujian	9
5	Penutup	10
6	Lampiran	11
6.1	Tautan GitHub	11
6.2	Tabel Kontribusi	11



Bab 1

Pendahuluan

Bab 2

Dasar Teori

2.1 Teori Singkat

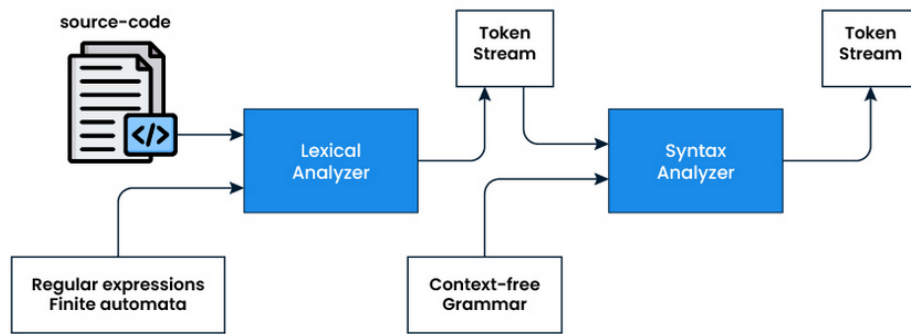
2.1.1 Parser dan Syntax Analysis

Parser atau syntax analyzer adalah komponen yang bertugas memeriksa struktur sintaks program berdasarkan aturan grammar bahasa. Setelah tahap lexical analysis selesai, parser menerima urutan token dan memverifikasi bahwa token-token tersebut dapat dibentuk menjadi konstruksi sintaks yang sah. Dengan kata lain, parser memeriksa apakah program mengikuti aturan bahasa, bukan lagi sekadar pola karakter.

Dalam pipeline kompilasi yang umum, posisi parser berada setelah lexer. Alur ini dapat diringkas sebagai berikut: source code diproses oleh lexer menjadi daftar token, kemudian parser mengolah token tersebut menjadi parse tree, yang selanjutnya dipakai oleh semantic analysis dan code generation. Parser tidak menerima source code mentah, melainkan daftar token yang sudah diklasifikasikan oleh lexer. Hubungan ini membuat output lexer pada Milestone 1 menjadi input utama parser pada Milestone 2.

Peran parser bersifat krusial karena ia menegakkan aturan grammar dan membangun representasi hierarkis dari program. Parser memeriksa urutan token apakah sesuai dengan produksi grammar, mendeteksi kesalahan sintaks, serta menyusun struktur parse tree yang menunjukkan bagaimana bagian-bagian program saling terhubung. Representasi ini menjadi dasar untuk analisis semantik dan tahapan kompilasi berikutnya.

Secara umum, parser dapat dibedakan menjadi pendekatan top-down dan bottom-up. Top-down memulai analisis dari simbol awal grammar lalu menurunkannya menjadi token yang cocok, sedangkan bottom-up membangun struktur dari token menuju simbol awal. Recursive Descent yang digunakan pada Arion Compiler adalah bentuk top-down parsing. Ketika terjadi syntax error, parser harus mampu memberikan pesan kesalahan yang informatif, misalnya menjelaskan token yang tidak sesuai dan posisi baris serta kolomnya agar proses debugging menjadi lebih efektif.

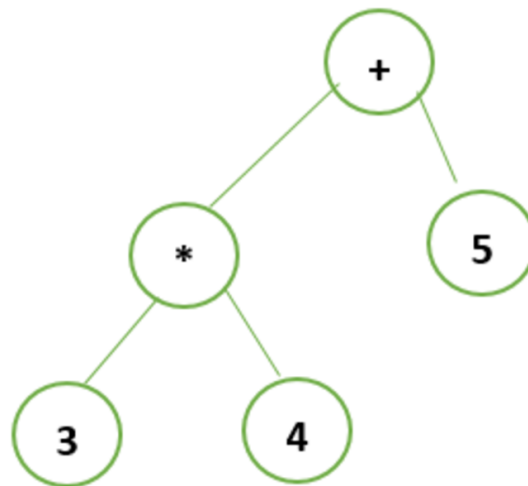


Gambar 2.1: Diagram pipeline kompilasi untuk syntax analyzer.

2.1.2 Parse Tree

Parse tree adalah representasi hierarkis dari struktur sintaks program berdasarkan grammar. Setiap simpul pada parse tree merepresentasikan penerapan aturan produksi sehingga hubungan antar bagian program dapat terlihat secara jelas. Parse tree menampilkan bagaimana sebuah program dibangun dari simbol awal hingga menghasilkan urutan token yang sesuai dengan grammar.

Dalam teori CFG, komponen parse tree dapat dijelaskan sebagai berikut. Node interior diberi label non-terminal, node daun diberi label terminal atau epsilon, dan root diberi label start symbol grammar. Setiap node interior A dengan anak $X_1, X_2, \dots, X_k$ menggambarkan produksi $A \rightarrow X_1X_2\dots X_k$. Dengan struktur ini, parse tree memberikan gambaran eksplisit dari proses derivasi yang menghasilkan program.

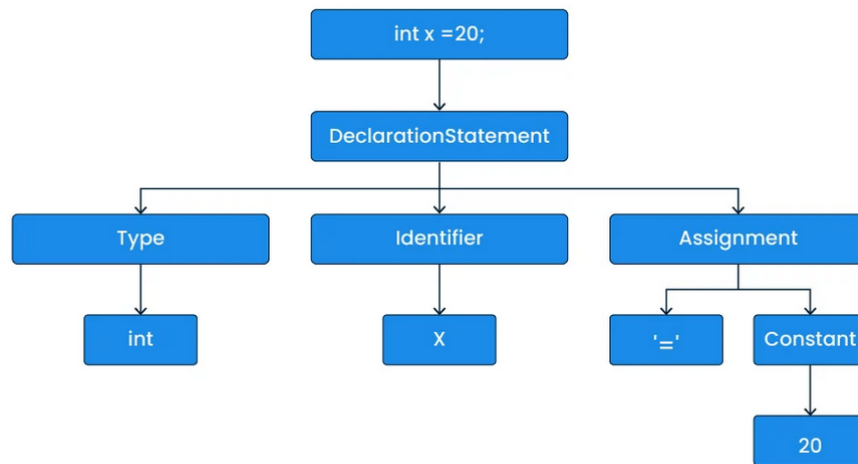


Gambar 2.2: Contoh syntax tree sederhana.

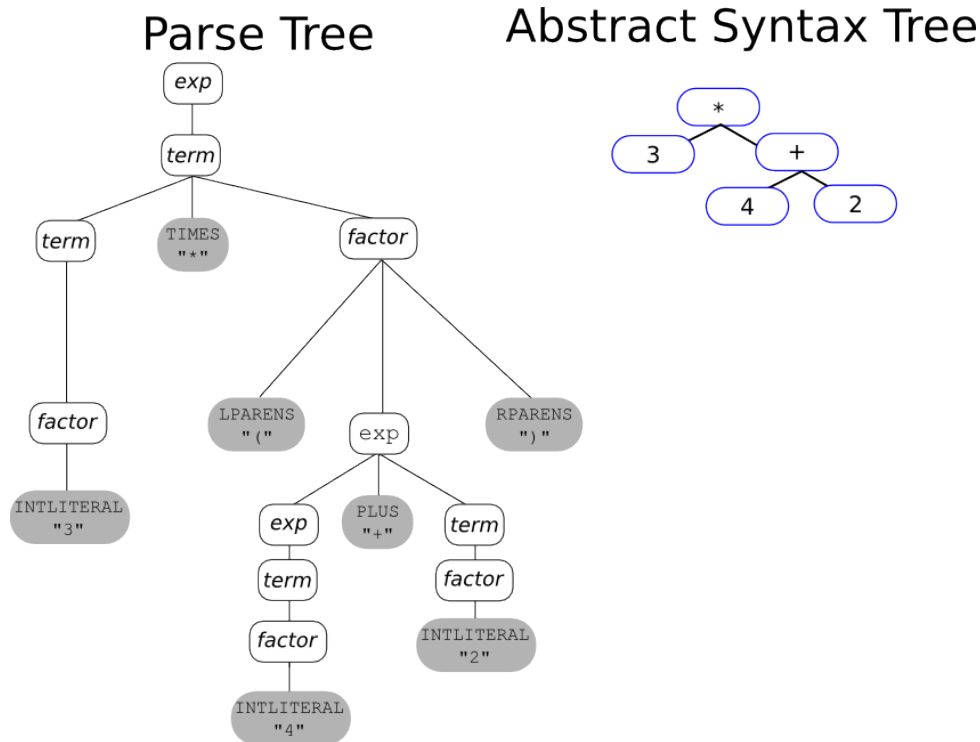
Konsep yield pada parse tree merujuk pada string terminal yang diperoleh de-

ngan membaca daun dari kiri ke kanan. Yield ini harus sama dengan urutan token input agar parse tree dianggap valid. Parse tree juga merupakan representasi alternatif dari derivasi leftmost atau rightmost, karena setiap derivasi tersebut dapat dipetakan ke satu parse tree yang setara secara struktural.

Parse tree berbeda dengan Abstract Syntax Tree (AST). Parse tree memuat semua simbol, termasuk tanda baca dan token terminal yang berfungsi sebagai struktur sintaks. AST mengabstraksi detail sintaks tersebut dan hanya menyimpan informasi semantis penting. Pada Milestone 2 ini, keluaran parser adalah parse tree lengkap, bukan AST. Parse tree penting karena menjadi input untuk semantic analysis dan menyediakan struktur program yang terverifikasi secara sintaks.



Gambar 2.3: Contoh parse tree untuk ekspresi aritmatika sederhana dengan label node terminal dan non-terminal.



Gambar 2.4: Perbandingan parse tree dan AST untuk ekspresi yang sama; parse tree lebih lengkap dengan semua terminal.

2.1.3 Recursive Descent Parser

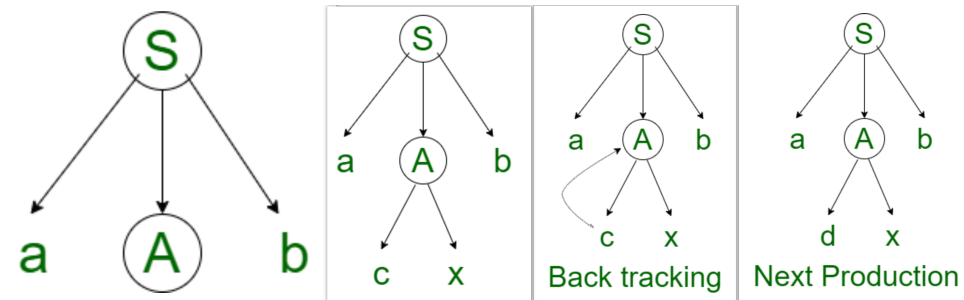
Recursive Descent Parser adalah metode parsing top-down yang mengimplementasikan satu fungsi rekursif untuk setiap non-terminal grammar. Ide utamanya adalah merepresentasikan aturan grammar sebagai prosedur yang secara langsung memeriksa token input dan membangun struktur parse tree. Pendekatan ini banyak digunakan karena sederhana dan mudah dipetakan ke kode program.

Pada top-down parsing, proses dimulai dari start symbol, lalu diperluas ke bawah mengikuti aturan produksi sampai mencapai terminal yang cocok dengan token input. Setiap fungsi non-terminal akan memeriksa token saat ini, memilih produksi yang relevan, lalu memanggil fungsi lain sesuai urutan simbol dalam produksi. Ketika sebuah terminal diharapkan, parser mencocokkan token input; jika cocok, token dikonsumsi dan parser maju ke token berikutnya.

Cara kerja recursive descent dapat dijelaskan langkah demi langkah. Setiap non-terminal memiliki fungsi `parseX()` yang membaca token, memanggil `parseY()` untuk non-terminal lain, dan mengonsumsi token ketika bertemu terminal yang diharapkan. Jika token tidak cocok, parser melempar syntax error. Fungsi-fungsi ini dipanggil secara rekursif sehingga struktur pemanggilan mengikuti struktur grammar. Contoh sederhana pada grammar ekspresi aritmatika adalah `parseExpression()` memanggil `parseTerm()`, dan `parseTerm()` memanggil `parseFactor()`.

Metode ini memiliki kelebihan karena mudah diimplementasikan dan struktur kodenya mencerminkan grammar. Namun, ia tidak dapat menangani grammar yang left-recursive secara langsung sehingga grammar harus ditransformasikan terlebih

dahulu. Selain itu, recursive descent umumnya bersifat deterministik dan mengandalkan lookahead terbatas, seperti LL(1). Dalam Arion Compiler, grammar bahasa di-hardcode ke dalam fungsi `parseX()` seperti `parseProgram()`, `parseDeclarationPart()`, `parseCompoundStatement()`, `parseStatement()`, dan `parseExpression()`, sehingga struktur parser menjadi jelas dan teruji.



Gambar 2.5: Ilustrasi proses recursive descent sebagai pohon pemanggilan fungsi selama parsing.



Bab 3

Perancangan dan Implementasi



Bab 4

Pengujian



Bab 5

Penutup

Bab 6

Lampiran

6.1 Tautan GitHub

Link GitHub

6.2 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Persentase Kontribusi (%)
1	Jonathan Kris Wicaksono	13524023	Membuat konsep algoritma program, menyusun <code>lexer.h</code> serta <code>lexer.c</code>	25
2	Vincent Rionarlie	13524031	Melakukan pembagian tugas, mendesain diagram transisi DFA	25
3	Muhammad Aufar Rizqi Kusuam	13524061	Menyusun laporan akhir	25
4	Bryan Pratama Putra Hendra	13524067	Mengkodekan <code>token.h</code> dan <code>token.c</code>	25